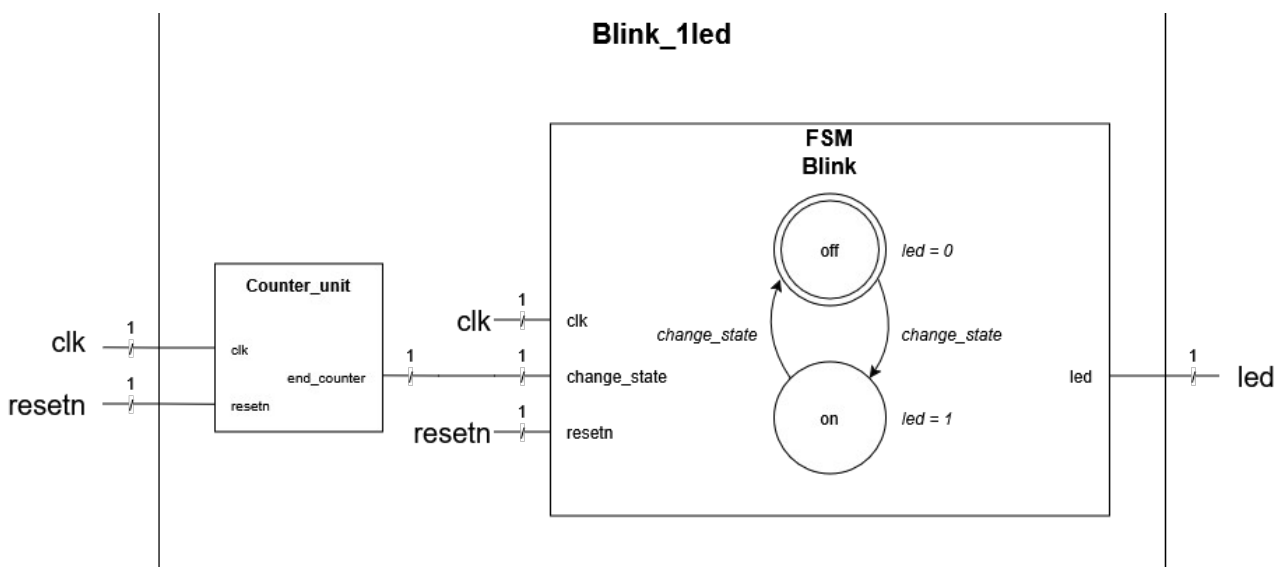


Formation FPGA

TP4 – Pilotage de LED et mémoire Partie 1

1) Voici le schéma RTL de notre architecture permettant de faire clignoter une LED :



Le module *Counter_unit* a été décrit initialement dans le TP2, c'est la version modifiée utilisée dans le TP3 qui est ici utilisée (voir fichier *counter_unit.vhd*) : lorsqu'il atteint un nombre de coups d'horloge défini par son paramètre *cible*, le compteur émet pendant un coup d'horloge un signal *end_counter*, et recommence à compter.

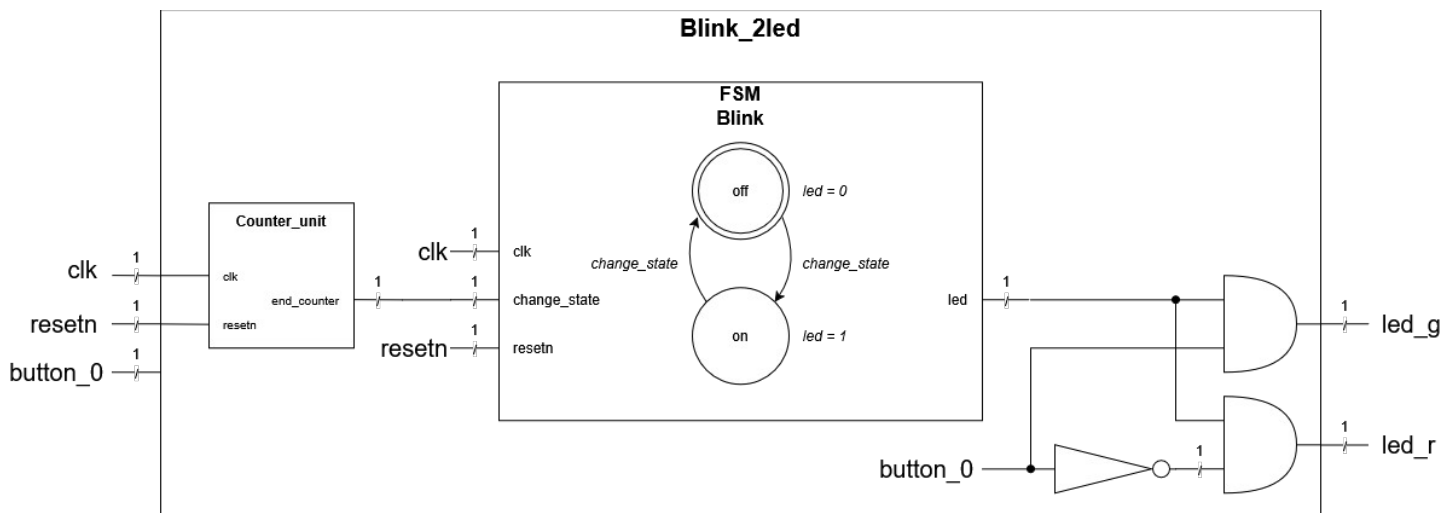
Ce module est relié à deux signaux d'entrée : un signal d'horloge *clk*, et un signal de reset externe asynchrone (en valeur haute) *resetn*.

La machine à état (FSM) ici appelée *Blink* permet de garder en mémoire l'état allumé ou éteint d'une unique LED, et passe d'un état à l'autre lorsque son entrée *change_state* est haute.

En reliant la sortie *end_counter* du module *Counter_unit* à l'entrée *change_state* de la FSM *Blink*, le clignotement est piloté par la période régulière du compteur.

La sortie de la FSM *Blink* est reliée à un signal de sortie de notre architecture pilotant une LED, nommé simplement *led* sur notre schéma RTL, mais qui pourrait par exemple être la sortie *led0_r* pour notre carte Cora Z7.

2) Nous modifions notre architecture RTL pour piloter maintenant 2 LED, sélectionnées en fonction d'un nouveau signal d'entrée *button_0*. En voici le nouveau schéma RTL :



On reconnaît le module *Counter_unit* et la FSM *Blink* décrits à la question précédentes, qui gèrent ensemble le rythme de clignotement des LED.

Cette fois, une logique combinatoire supplémentaire se trouve entre la sortie *led* de la FSM *Blink* et les signaux de sorties de LED. Deux portes AND permettent de comparer le signal *led* et un nouveau signal d'entrée *button_0* :

- Lorsque *led* et *button_0* sont tous les deux hauts, le signal de sortie *led_g* pilotant une LED verte est activé.
- Lorsque *led* est haut et *button_0* est bas (et donc que ce dernier est haut après une porte NOT), le signal de sortie *led_r* pilotant une LED rouge est activé.

3) L'architecture décrite dans le schéma RTL précédent est retranscrite en code VHDL, définissant un module *Blink_2led* à l'intérieur du nouveau fichier VHDL *blink_2led.vhd*.

En plus des entrées et sorties visibles sur le schéma, on définit pour notre module un paramètre *generic()* : un paramètre *cible* (stocké comme positive, sur 31 bits), qui est utilisé pour la définition du paramètre éponyme dans l'instance du *Counter_unit* interne, et qui va donc définir la durée en nombre de périodes d'horloge des cycles qu'on va compter.

4) On établit un testbench du module *Blink_2led* à l'intérieur du fichier de simulation *tb_blink_2led.vhd*.

Pour rendre notre simulation rapide, on va repartir des paramètres utilisés dans la première simulation du TP3 :

- Une horloge de fréquence 100 MHz, soit une période de 10 ns, avec un signal carré (demi-période 5 ns)
- Une *cible* du compteur de 200000, ce qui correspond à des cycles de 2 ms

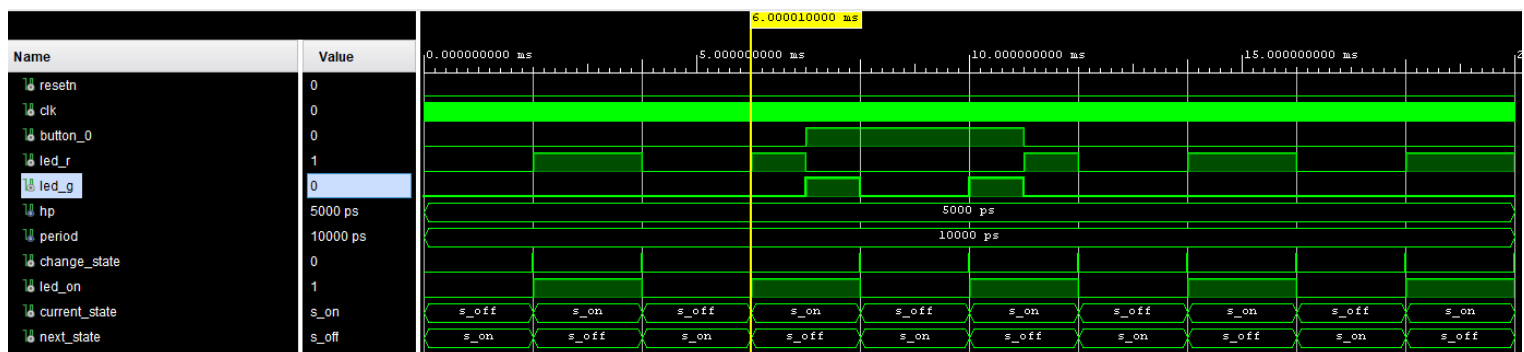
Pendant notre simulation, on va observer principalement :

- Nos deux sorties qui pilotent les LEDs (*led_r* et *led_g*).
- L'entrée *button_0*, qui doit faire passer la LED qui clignote de rouge vers le vert quand elle est appuyée, et inversement quand elle est relâchée.

On peut aussi observer quelques signaux internes pour vérifier le bon fonctionnement :

- Le signal *current_state*, qui indique l'état actuel de la FSM *Blink*.
- Le signal *next_state*, qui indique le prochain état de la FSM *Blink*.
- Le signal *change_state* qui indique un changement d'état de la FSM *Blink*, lorsque le module *Counter_unit* atteint sa cible.
- Le signal *led_on* qui gère le clignotement global (pas la couleur).

Voici le chronogramme obtenu pour 20 ms de simulation, avec une activation du signal *button_0* entre t=7 ms et t=11 ms :



Pendant les 7 premières millisecondes de fonctionnement normal, on observe bien un pic de *change_state* toutes les 2 millisecondes, suivi à chaque fois d'un changement d'état de la FSM *Blink*, qui se traduit par une alternance entre les états hauts et bas du signal *led_on*, et donc du signal de la LED rouge *led_r* (sur cette période, le bouton n'est jamais activé).

À t=7 ms, le signal *button_0* est activé. On observe comme attendu qu'à ce moment, la couleur de la LED allumée change du rouge vers le vert : *led_r* se désactive et *led_g* s'active.

Le clignotement continue en vert, sans décalage à partir de là : à t=8 ms, le signal *led_g* se désactive, puis se réactive à t=10 ms.

À t=11 ms, le signal *button_0* est désactivé. La couleur de la LED allumée change à nouveau, cette fois de vert vers rouge : *led_g* se désactive et *led_r* s'active.

Le clignotement reprend en rouge à partir de là, et jusqu'à la fin de la simulation.

On a également implémenté des tests automatiques dans notre testbench, pour vérifier que le comportement du module suit bien ce que nous attendons. Tous ces tests ont été vérifiés :

```
Note: test OK, only led_r = '1' at 2.000010 ms
Time: 2000010 ns Iteration: 0 Process: /tb_blin
Note: test OK, all led = '0' at 4.000010 ms
Time: 4000010 ns Iteration: 0 Process: /tb_blin
Note: test OK, only led_r = '1' at 7 ms
Time: 7 ms Iteration: 0 Process: /tb_blink_2le
Note: test OK, only led_g = '1' at 7.000010 ms
Time: 7000010 ns Iteration: 0 Process: /tb_blin
Note: test OK, all led = '0' at 8.000010 ms
Time: 8000020 ns Iteration: 0 Process: /tb_blin
Note: test OK, only led_g = '1' at 11 ms
Time: 11000010 ns Iteration: 0 Process: /tb_blin
Note: test OK, only led_r = '1' at 11.000010 ms
Time: 11000020 ns Iteration: 0 Process: /tb_blin
Note: test OK, all led = '0' at 12.000010 ms
Time: 12000020 ns Iteration: 0 Process: /tb_blin
```

Comme on l’a observé dans notre simulation où le bouton est pressé à cheval sur trois cycles du module *Counter_unit* différents, tant que le bouton est pressé on continue a clignoter en vert dans les cycles de clignotements successifs.

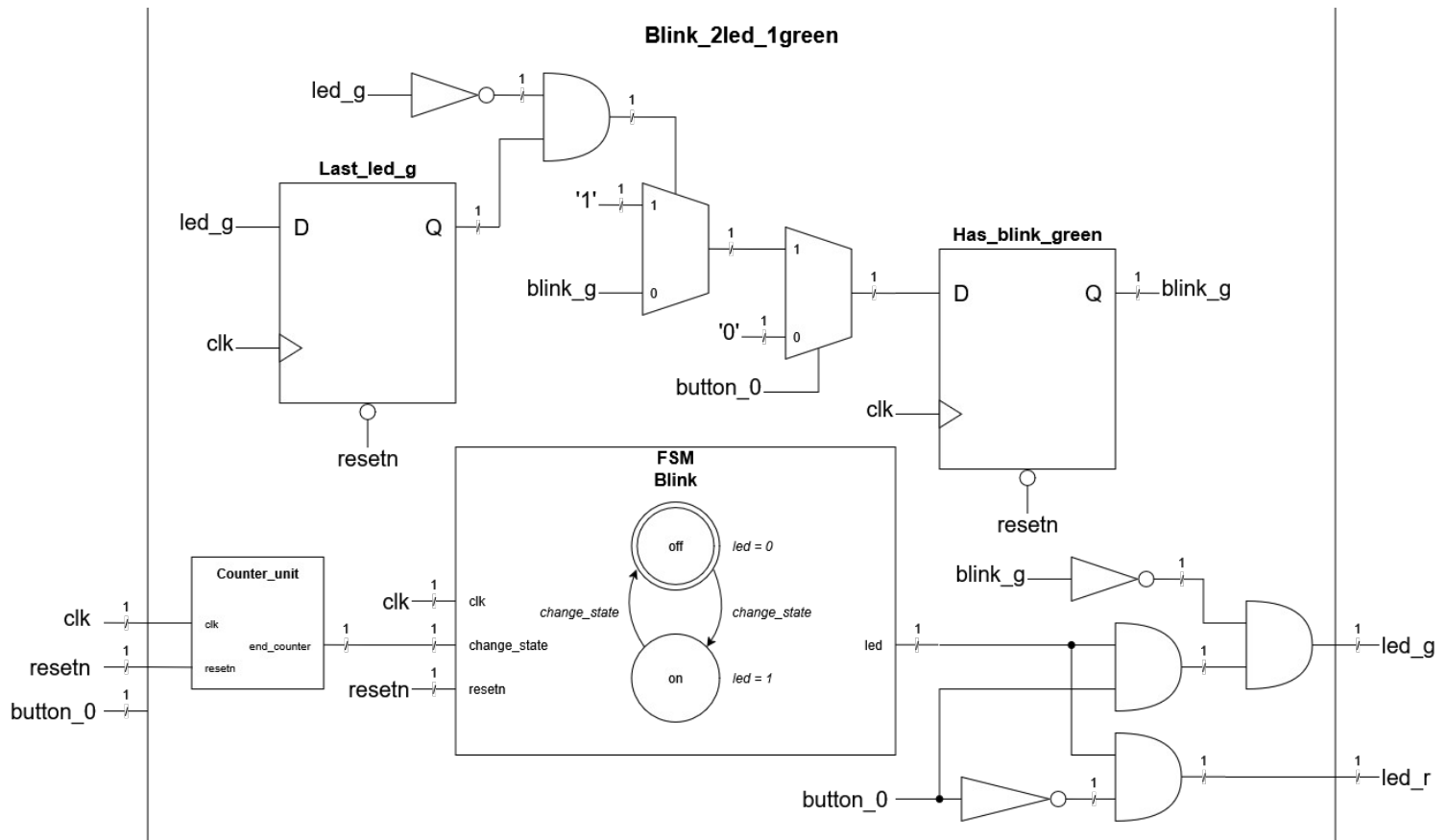
5) Pour que la LED ne clignote en vert qu’une seule fois même si le bouton est maintenu, il faudrait que notre système garde en mémoire si il a déjà fini de clignoter une fois depuis que le bouton a été appuyé.

Cette information, qu’on va appelé *blink_g*, pourrait être stockée dans un registre (relié au même signaux d’horloge et de reset que le reste de l’architecture), qui serait remis à ‘0’ lorsque le bouton est relâché, et qui passerait à ‘1’ lorsque la LED verte finis de clignoter, c’est à dire sur un front descendant du signal *led_g*. *blink_g* reste inchangé dans les autres cas.

La détection de ce front descendant se fait avec un registre supplémentaire (même horloge et reset) qui enregistre l’état au coup d’horloge précédent de *led_g* : un front descendant a lieu quand l’ancienne valeur vaut ‘1’ alors que la nouvelle vaut ‘0’.

La valeur de *blink_g* doit être ajoutée à la logique combinatoire qui décide de la valeur de *led_g* : pour que ce signal soit activé, en plus de nécessiter que *led* et *button_0* soit haut, il faut que *blink_g* soit bas (haut après une porte NOT)

6) Voici le schéma RTL actualisé de notre architecture pour ajouter cette nouvelle condition :



7) La version mise-à-jour de notre architecture est implémentée dans le module *Blink_2led_1green* à l'intérieur d'un nouveau fichier VHDL *blink_2led_1green.vhd*.

On établit un nouveau testbench pour le module *Blink_2led_1green* à l'intérieur du fichier de simulation *tb_blink_2led_1green.vhd*.

On reprends les paramètres de simulation de la simulation précédente (horloge à 100 MHz, *cible* équivalent à 2 ms).

Encore une fois, on va observer principalement :

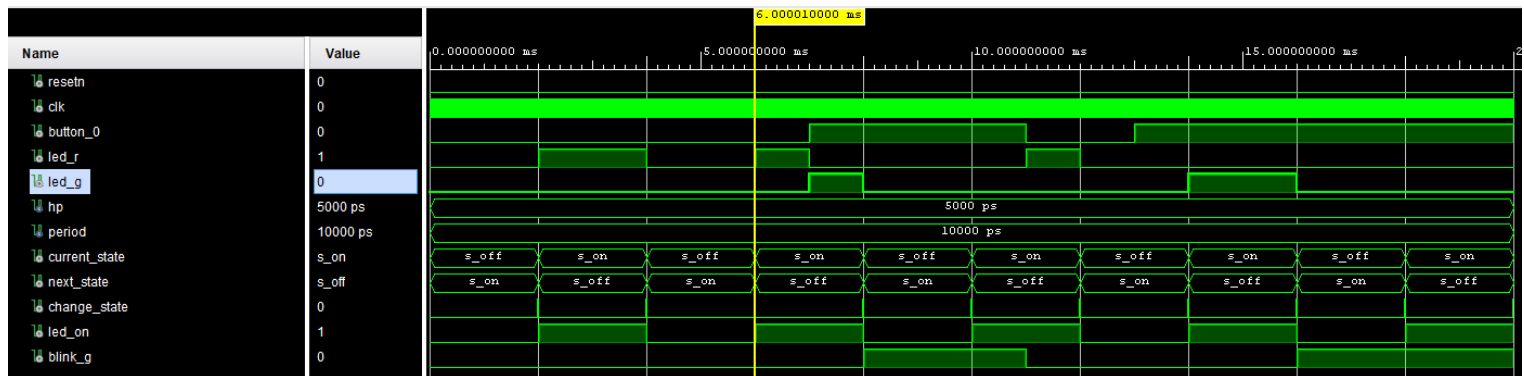
- Nos deux sorties qui pilotent les LEDs (*led_r* et *led_g*).
- L'entrée *button_0*, qui doit faire passer la LED qui clignote de rouge vers le vert pour un unique clignotement quand elle est appuyée, et repasser au rouge quand elle est relâchée.

On va aussi observer quelques signaux internes pour vérifier le bon fonctionnement, les premiers étaient déjà présent dans la simulation précédent, mais le dernier est propre à cette nouvelle version :

- Le signal *current_state*, qui indique l'état actuel de la FSM *Blink*.
- Le signal *next_state*, qui indique le prochain état de la FSM *Blink*.

- Le signal *change_state* qui indique un changement d'état de la FSM *Blink*, lorsque le module *Counter_unit* atteint sa cible.
- Le signal *led_on* qui gère le clignotement global (pas la couleur).
- Le signal *blink_g* qui s'active après une fin de clignotement en vert et se réinitialise au relâchement du bouton, qui empêche les clignotements en verts après le premier.

Voici le chronogramme obtenu pour 20 ms de simulation, avec une activation du signal *button_0* entre $t=7$ ms et $t=11$ ms, puis de $t=13$ ms jusqu'à la fin de la simulation :



Pendant les 7 premières millisecondes de fonctionnement normal, on observe comme précédemment un pic de *change_state* toutes les 2 millisecondes, suivi à chaque fois d'un changement d'état de la FSM *Blink*, qui se traduit par une alternance entre les états hauts et bas du signal *led_on*, et donc du signal de la LED rouge *led_r* (sur cette période, le bouton n'est jamais activé).

À $t=7$ ms, le signal *button_0* est activé. On observe comme attendu qu'à ce moment, la couleur de la LED allumée change du rouge vers le vert : *led_r* se désactive et *led_g* s'active.

Ce clignotement en vert se termine, sans décalage, à $t=8$ ms, le signal *led_g* se désactive, un front descendant est détecté et le signal *blink_g* s'active.

À $t=10$ ms, un nouveau cycle de clignotement devrait commencer (*led_on* passe à '1'), mais comme on a déjà eu un clignotement en vert depuis que le bouton est appuyé (i.e. *blink_g* vaut '1') la LED verte ne s'allume pas.

À $t=11$ ms, le signal *button_0* est désactivé, sans effet particulier. On repasse sur un fonctionnement classique avec la LED rouge, et puisqu'on est toujours au milieu d'un cycle de clignotement où on devrait être allumé (*led_on* vaut '1'), *led_r* s'active. On note aussi qu'à ce moment, avec le relâchement du bouton, *blink_g* repasse à '0'.

À $t=13$ ms, le signal *button_0* est réactivé. Comme on est à ce moment sur une période du clignotement où la LED doit être éteinte (*led_on* vaut '0'), le signal *led_g* reste désactivé.

À $t=14$ ms, on commence un nouveau cycle de clignotement allumé, et puisqu'il n'y a eu aucun clignotement depuis la pression du bouton (*blink_g* vaut '0'), le signal *led_g* s'active.

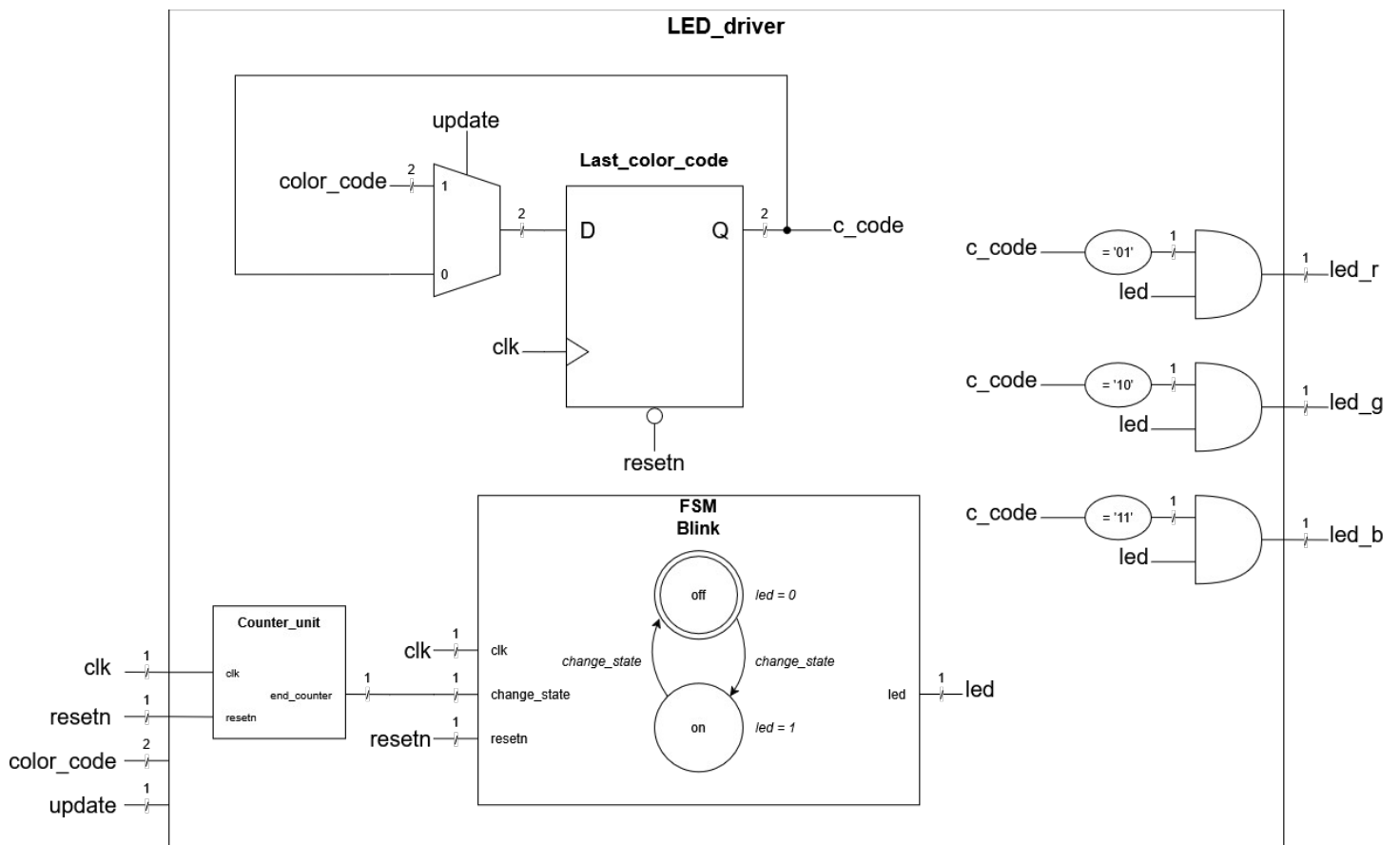
Ce clignotement en vert se termine à $t=16$ ms, le signal *led_g* se désactive, un front descendant est détecté et le signal *blink_g* s'active.

À $t=18$ ms, un nouveau cycle de clignotement devrait commencer (*led_on* passe à '1'), mais comme on a déjà eu un clignotement en vert depuis que le bouton est appuyé (i.e. *blink_g* vaut '1') la LED verte ne s'allume pas.

On a également implémenté des tests automatiques dans notre testbench, pour vérifier que le comportement du module suit bien ce que nous attendons. Tous ces tests ont été vérifiés :

```
Note: test OK, only led_r = '1' at 2.000010 ms
Time: 2000010 ns Iteration: 0 Process: /tb_blink
Note: test OK, all led = '0' at 4.000010 ms
Time: 4000010 ns Iteration: 0 Process: /tb_blink
Note: test OK, only led_r = '1' at 7 ms
Time: 7 ms Iteration: 0 Process: /tb_blink_2led_
Note: test OK, only led_g = '1' at 7.000010 ms
Time: 7000010 ns Iteration: 0 Process: /tb_blink
Note: test OK, all led = '0' at 8.000010 ms
Time: 8000020 ns Iteration: 0 Process: /tb_blink
Note: test OK, all led = '0' at 10.000010 ms
Time: 10000020 ns Iteration: 0 Process: /tb_blin
Note: test OK, all led = '0' at 11 ms
Time: 11000010 ns Iteration: 0 Process: /tb_blin
Note: test OK, only led_r = '1' at 11.000010 ms
Time: 11000020 ns Iteration: 0 Process: /tb_blin
Note: test OK, all led = '0' at 12.000010 ms
Time: 12000020 ns Iteration: 0 Process: /tb_blin
Note: test OK, only led_g = '1' at 14.000010 ms
Time: 14000020 ns Iteration: 0 Process: /tb_blin
Note: test OK, all led = '0' at 18.000010 ms
Time: 18000020 ns Iteration: 0 Process: /tb_blin
```

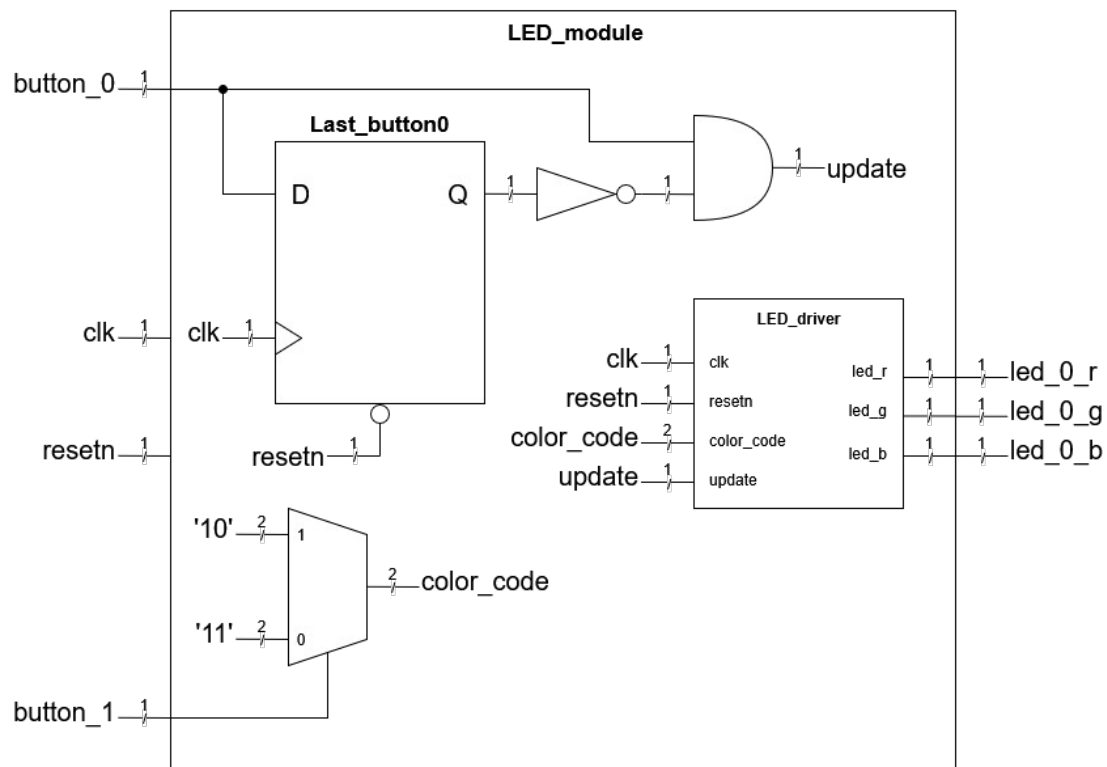
8) Voici l'architecture RTL correspondant au module *LED_driver* qui fait clignoter une LED RGB selon un code couleur *color_code*, mis à jour lorsqu'un signal *update* est reçu :



L'architecture RTL comporte :

- Une instantiation de *Counter_unit* associée à la FSM *Blink* décrite précédemment pour générer le signal global de clignotement de la LED, *led*.
- La partie qui gère la mémorisation et le stockage du code couleur utilisé (signal interne *c_code*) :
 - Un registre (dirigé par l'horloge *clk* et le signal de reset haut *resetn*) sur 2 bits qui stocke le signal *c_code* ;
 - Un multiplexeur qui gère l'actualisation du registre, en prenant le signal d'entrée *color_code* lorsque le signal d'entrée *update* est haut, et en gardant le code précédent sinon.
- La logique combinatoire, qui gère l'allumage de chaque couleur de la LED. On a pour chaque couleur une porte AND, qui allume la LED si le signal *led* est activé, et si le signal *color_c* correspond à la valeur pour cette couleur.

9) Voici l'architecture de notre module *LED_module* complet, avec pilotage des entrées et sorties du module *LED_driver* :



Cette architecture RTL comporte :

- La création du signal *color_code*, en fonction de l'état du signal d'entrée *button_1*. Grâce à un multiplexeur, *color_code* vaut '10' (couleur verte) quand le bouton 1 est appuyé, et '11' (couleur bleue) quand le bouton 1 est relâché.
- La génération du signal *update*, avec une détection de front montant sur le signal *button_0*. Cette détection est effectuée avec :
 - Un registre qui enregistre la valeur précédente de *button_0* dans le signal interne *last_button0*.
 - Une porte AND, qui détecte le front montant lorsque le signal *button_0* vaut '1' alors que le signal précédent *last_button0* vaut '0'.
- Une instance du module *LED_driver*, dont les entrées sont les signaux d'entrée *clk* et *reset*, ainsi que les signaux générés aux étapes précédentes *color_code* et *update*. Les sorties du module sont liées aux couleurs respectives de la LED 0 (*led_0_r*, *led_0_g* et *led_0_b*).

10) On implémente l'architecture RTL décrite à la question précédente dans du code VHDL.

En plus du code du module *Counter_unit* décrit à la question 1, nous séparons notre code dans deux fichiers différents.

D'abord le fichier *LED_driver.vhd*, où l'on décrit le module *LED_driver* de la question 8. Ce code appelle une instance du module *Counter_unit*, et possède :

- Une partie séquentiel, qui gère :
 - Le reset des registres (*c_code* et l'état de la FSM *Blink*)
 - Le changement d'état de la FSM
 - La mise à jour du registre *c_code*
 - Le signal *led_on* en fonction de l'état de la FSM
- Une partie combinatoire qui gère l'allumage individuel des couleurs de LED en fonction du signal *led_on* et du code couleur actuel *c_code*.

À noter par rapport au schéma RTL : on a désigné une valeur par défaut pour le signal interne *c_code* à '01' (rouge), qui est affectée à l'initialisation et après un reset. Ainsi, tant que le module n'a pas reçu un premier signal *update*, il va clignoter en rouge.

Ensuite, nous décrivons le module global *LED_module* dans le fichier *LED_module.vhd*. Ce code appelle une instance du module *LED_driver*, et possède :

- Une partie séquentiel, qui gère le reset et la mise à jour du registre *last_button0*.
- Une partie combinatoire qui gère :
 - Le signal *update* sur le front montant du bouton 0
 - Le signal *color_code* en fonction de l'état du bouton 1

11) On établit un nouveau testbench pour le module *LED_module* à l'intérieur du fichier de simulation *tb_LED_module.vhd*.

On reprends les paramètres de simulation de la simulation précédente pour l'horloge à 100 MHz, mais pour cette simulation on choisit un paramètre *cible* équivalent à 1 ms.

12) Pendant notre simulation, on va observer principalement :

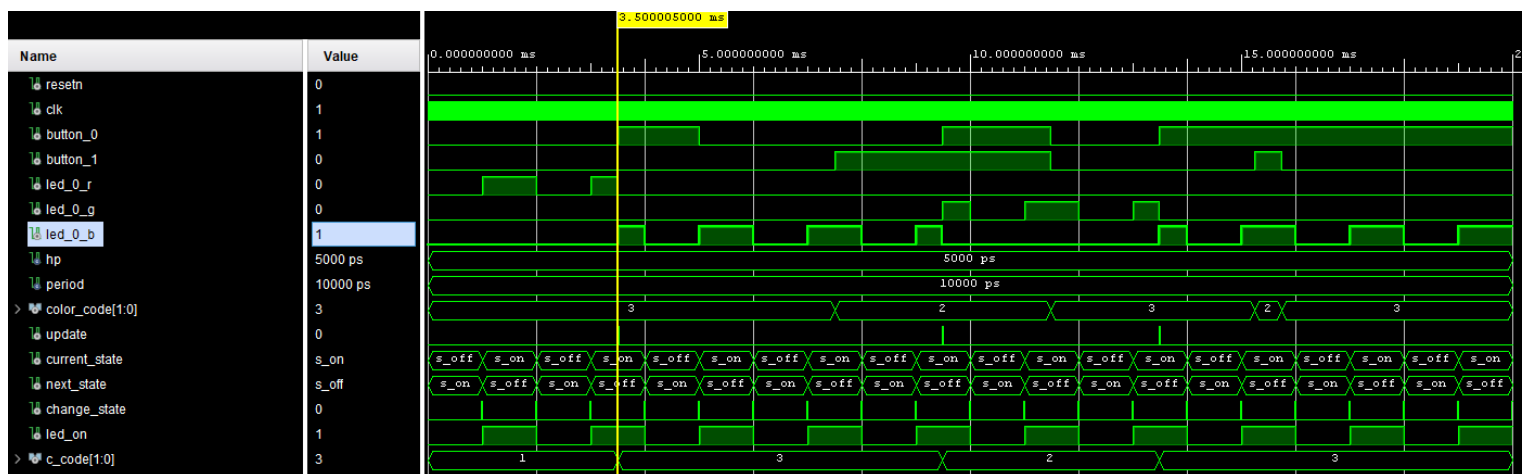
- Nos trois sorties qui pilotent les LEDs (*led_0_r*, *led_0_g* et *led_0_b*)
- L'entrée *button_0*, qui gère le signal *update* et active un éventuel changement de couleur de la LED.
- L'entrée *button_1*, qui gère le signal *color_code*.

On va aussi observer quelques signaux internes pour vérifier le bon fonctionnement :

- Les signaux *current_state* et *next_state*, qui indiquent respectivement les états actuel et futur de la FSM *Blink*.

- Le signal *change_state* qui déclenche un changement d'état de la FSM *Blink* (lorsque le module *Counter_unit* atteint sa cible).
- Le signal *led_on* qui gère le clignotement global (pas la couleur).
- Le signal *c_code* qui indique la couleur actuelle dans laquelle la LED clignote.
- Les signaux *color_code* et *update* qui gère la mise-à-jour de *c_code*, le premier indiquant la nouvelle valeur et le second déclenchant la mise-à-jour

Voici le chronogramme obtenu pour 20 ms de simulation, avec une activation du signal *button_0* entre $t=3,5$ ms et $t=5$ ms, puis de $t=9,5$ ms à $t=11,5$ ms et finalement $t=13,5$ ms jusqu'à la fin de la simulation, et une activation du signal *button_1* entre $t=7,5$ ms et $t=11,5$ ms puis entre $t=15,25$ ms et $t=15,75$ ms :



De manière générale, on observe que le mécanisme de clignotement (dans le schéma RTL, le module *Counter_unit* et la FSM *Blink*) produise bien un signal *led_on* qui alterne entre ses états bas et haut toutes les secondes, avec un signal *change_state* faisant un pic à chaque seconde, et l'état de la FSM (*current_state* et *next_state*) qui alterne après chacun de ces pics.

Jusqu'à $t=3,5$ ms, avant toute pression des boutons, on observe un clignotement de la LED rouge.

À $t=3,5$ ms, on appuie sur le bouton 0, un front montant est détecté ce qui active le signal *update*. Au coup d'horloge suivant, cela déclenche donc la mise-à-jour du code couleur interne *color_c*, qui prend la valeur de *color_code* à '11' puisque le bouton 1 n'est pas appuyé. La couleur de la LED en train de clignoter change donc, passant de rouge à bleu.

Le signal du bouton 0 n'ayant un effet que sur front montant, le clignotement continue normalement en bleu pendant que le bouton est appuyé, et sans changement lorsqu'il est relâché à $t=5$ ms.

À $t=7,5$ ms, le bouton 1 est appuyé, et on voit que la valeur de *color_code* change bien à '10'. Cependant, en l'absence de signal *update*, la valeur de *color_c* reste inchangée et le clignotement continue en bleu.

À $t=9,5$ ms, on appuie à nouveau sur le bouton 0, et le signal *update* s'active. Au coup d'horloge suivant, cela déclenche donc la mise-à-jour du code couleur interne *color_c*, qui prend la valeur de *color_code* à '10'. La couleur de la LED en train de clignoter change donc, passant de bleu à vert.

À $t=11,5$ ms, on relâche les deux boutons 0 et 1. Cela n'a aucun effet sur le signal *update*, donc la couleur de clignotement ne change pas, et la valeur de *color_code* repasse à '11'.

À $t=13,5$ ms, on appuie encore sur le bouton 0, et le signal *update* s'active à nouveau. Au coup d'horloge suivant, cela déclenche la mise-à-jour du code couleur interne *color_c*, qui prend la valeur de *color_code* à '11'. La couleur de la LED en train de clignoter change donc, repassant de vert à bleu.

Entre $t=15,25$ ms et $t=15,75$ ms, on appuie sur le bouton 1. Pendant cet intervalle, la valeur de *color_code* passe à '10'. En l'absence de signal *update* (le bouton 0 reste appuyé sur toute la période), il n'y a cependant aucun changement sur la couleur de la LED en train de clignoter, qui reste bleue.

Jusqu'à la fin de la simulation, sans aucun changement sur les boutons, la LED continue de clignoter en bleu.

On a encore une fois implémenté des tests automatiques dans notre testbench, pour vérifier que le comportement de notre module suit bien ce que nous attendons. Tous ces tests ont été vérifiés :

```
Note: test OK, only led_0_r = '1' at 1.000010 ms
Time: 1000010 ns Iteration: 0 Process: /tb_LED_mo
Note: test OK, all led = '0' at 2.000010 ms
Time: 2000010 ns Iteration: 0 Process: /tb_LED_mo
Note: test OK, only led_0_r = '1' at 3.5 ms
Time: 3500 us Iteration: 0 Process: /tb_LED_modul
Note: update : button_0 pushed at 3.5 ms
Time: 3500 us Iteration: 0 Process: /tb_LED_modul
Note: test OK, only led_0_b = '1' at 3.500010 ms
Time: 3500010 ns Iteration: 0 Process: /tb_LED_mo
Note: test OK, all led = '0' at 4.000010 ms
Time: 4000010 ns Iteration: 0 Process: /tb_LED_mo
Note: update : button_0 released at 5 ms
Time: 5 ms Iteration: 0 Process: /tb_LED_module/1
Note: test OK, only led_0_b = '1' at 5.000010 ms
Time: 5000010 ns Iteration: 0 Process: /tb_LED_mo
Note: test OK, only led_0_b = '1' at 7.5 ms
Time: 7500 us Iteration: 0 Process: /tb_LED_modul
Note: color_code : button_1 pushed at 7.5 ms
Time: 7500 us Iteration: 0 Process: /tb_LED_modul
Note: test OK, only led_0_b = '1' at 7.500010 ms
Time: 7500 us Iteration: 0 Process: /tb_LED_modul
Note: test OK, only led_0_b = '1' at 9.5 ms
Time: 9499990 ns Iteration: 0 Process: /tb_LED_mo
Note: update : button_0 pushed at 9.5 ms
Time: 9499990 ns Iteration: 0 Process: /tb_LED_mo
Note: test OK, only led_0_g = '1' at 9.500010 ms
Time: 9500 us Iteration: 0 Process: /tb_LED_modul
Note: test OK, only led_0_g = '1' at 11.5 ms
Time: 11499990 ns Iteration: 0 Process: /tb_LED_m
```

```
Note: update : button_0 released at 11.5 ms
Time: 11499990 ns Iteration: 0 Process: /tb_LED_m
Note: color_code : button_1 released at 11.5 ms
Time: 11499990 ns Iteration: 0 Process: /tb_LED_m
Note: test OK, only led_0_g = '1' at 11.500010 ms
Time: 11500 us Iteration: 0 Process: /tb_LED_modu
Note: test OK, only led_0_g = '1' at 13.5 ms
Time: 13499990 ns Iteration: 0 Process: /tb_LED_m
Note: update : button_0 pushed at 13.5 ms
Time: 13499990 ns Iteration: 0 Process: /tb_LED_m
Note: test OK, only led_0_b = '1' at 13.500010 ms
Time: 13500 us Iteration: 0 Process: /tb_LED_modu
Note: update : button_0 released at 14 ms
Time: 13999990 ns Iteration: 0 Process: /tb_LED_m
Note: test OK, only led_0_b = '1' at 15.25 ms
Time: 15249990 ns Iteration: 0 Process: /tb_LED_m
Note: color_code : button_1 pushed at 15.25 ms
Time: 15249990 ns Iteration: 0 Process: /tb_LED_m
Note: test OK, only led_0_b = '1' at 15.250010 ms
Time: 15250 us Iteration: 0 Process: /tb_LED_modu
Note: color_code : button_1 released at 15.75 ms
Time: 15749990 ns Iteration: 0 Process: /tb_LED_m
Note: test OK, only led_0_b = '1' at 15.750010 ms
Time: 15750 us Iteration: 0 Process: /tb_LED_modu
Note: test OK, only led_0_b = '1' at 17.5 ms
Time: 17249990 ns Iteration: 0 Process: /tb_LED_m
Note: update : button_0 pushed at 17.5 ms
Time: 17249990 ns Iteration: 0 Process: /tb_LED_m
Note: test OK, only led_0_b = '1' at 17.500010 ms
Time: 17249990 ns Iteration: 0 Process: /tb_LED_m
```

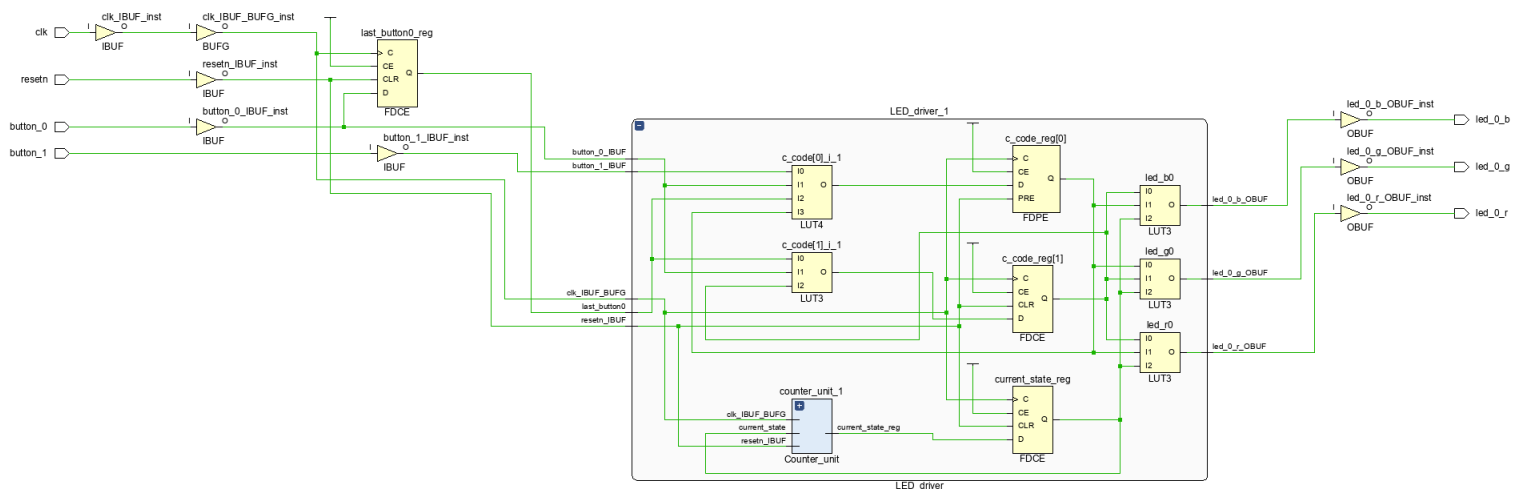
13) On réalise une première synthèse de notre code sur Vivado, en utilisant la stratégie « Vivado Synthesis Defaults ».

Le rapport de synthèse (disponible dans son entièreté dans le fichier *LED_module_synth1.vds*) nous donne les ressources utilisées suivantes :

Report Cell Usage:

	Cell	Count
1	BUFG	1
2	CARRY4	7
3	LUT3	17
4	LUT4	5
5	LUT5	2
6	LUT6	5
7	FDCE	30
8	FDPE	1
9	IBUF	4
10	OBUF	3

Pour avoir une meilleure compréhension de l'utilisation de ces ressources par rapport à notre architecture RTL, observons le schéma d'implémentation :



Les sous-divisions de ce schéma sont légèrement trompeuse, puisque sont regroupées dans un groupe appelé *counter_unit_1* non seulement l'ensemble des primitives servant à synthétiser l'instance de *Counter_unit*, mais également une partie de la logique combinatoire de la FSM *Blink*.

La version entièrement développée de ce schéma, nécessaire pour comprendre l'usage des ressources pour le module *Counter_unit*, est disponible dans le fichier *Schematique_TP4-1_synth1.pdf*.

Commençons par ce qui est visible sur le schéma non développé :

- Les quatre signaux d'entrée nécessitent chacun un buffer d'entrée **IBUF** (*clk_IBUF_inst*, *resetn_IBUF_inst*, *button_0_IBUF_inst* et *button_1_IBUF_inst*) soit **4 IBUF** au total, et un buffer d'horloge **BUFG** (*clk_IBUF_BUFG_inst*).
- Un registre FDCE (*last_button0_reg*) permet de stocker la valeur au coup d'horloge précédent de l'entrée *button_0*.
- Pour l'actualisation du signal *c_code* :
 - Une table de vérité à 4 entrées LUT4 (*c_code[0]_i_1*) est utilisée pour la valeur du dernier bit : 2 des entrées servent à détecter un front montant sur *button_0* (grâce à sa valeur au coup d'horloge précédent), lorsque le front montant est détecté (cela revient à un signal *update*) la nouvelle valeur dépend de l'état de *button_1*, lorsqu'il n'y a pas de front montant la nouvelle valeur est identique à l'ancienne.
 - Une table de vérité à 3 entrées LUT3 (*c_code[1]_i_1*) est suffisante pour la valeur du premier bit, qui lui ne dépend pas de l'état de *button_1* : lorsqu'on a un front montant sur *button_0* la valeur du bit passe à '1', et sans front montant la valeur reste identique à l'ancienne.
- Deux registres sont utilisés pour stocker la valeur de *color_c* :
 - Le dernier bit est stocké sur un registre **FDPE** (*c_code_reg[0]*) qui a la particularité de prendre une valeur haute au lieu de basse en cas de reset.
 - Le premier bit est stocké sur un registre classique FDCE (*c_code_reg[1]*).
- L'état actuel de la FSM *current_state* est stocké dans un registre classique FDCE (*current_state_reg*).
- L'état de chacune des couleurs de LED est déterminé par une table de vérité à trois entrées LUT3 (*led_b0*, *led_g0* et *led_r0*), soit 3 LUT3 au total, chacune fonctionnant avec les deux bits de *c_code* et la valeur de *current_state*.
- Les trois signaux de sortie nécessitent chacun un buffer de sortie **OBUF** (*led_0_b_OBUF_inst*, *led_0_g_OBUF_inst* et *led_0_r_OBUF_inst*), soit **3 OBUF** au total.

Examinons ensuite la sous-partie contenue dans le groupe *counter_unit_1*. On remarquera que le fonctionnement correspondant au module *Counter_unit*, décrit dans le TP2 et à nouveau observé dans le TP3, est maintenant optimisé d'une façon assez différente.

Je fais l'hypothèse que ce changement de fonctionnement est du à un changement de bibliothèque VHDL, puisque j'ai remplacé la bibliothèque *std_logic_unsigned* par *numeric_std* à la fin de mon TP3 pour suivre les suggestions de la correction du TP2. Le schéma de synthèse présenté dans mon rapport correspond à l'ancienne bibliothèque, et je n'avais à l'époque remarqué aucune différence de simulation ou de fonctionnement après remplacement de la bibliothèque, mais j'ai remarqué après le rendu du rapport que mon schéma de synthèse avait en fait évolué depuis le changement.

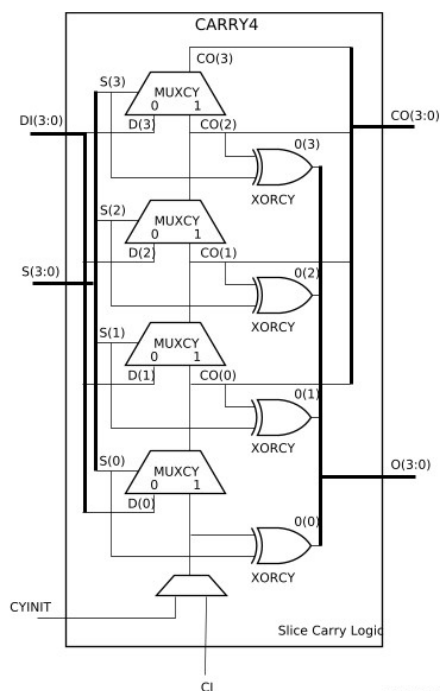
Notre paramètre *cible* vaut la valeur par défaut 125000000, soit 1 s pour l'horloge à 125 MHz de notre Cora Z7, notre signal compteur *Reg_ctr* comporte donc 27 bits.

Voici le détail du nouveau fonctionnement :

- Les 27 bits du signal compteur *Reg_ctr* sont stockés dans 27 registres FDCE (*Reg_ctr_reg[0]* à *Reg_ctr_reg[26]*).
- La comparaison de *Reg_ctr* avec le paramètre *cible* est effectuée grâce à un total de 7 tables de vérités LUT (*current_state_i_2* à *current_state_i_7* ainsi que *Reg_ctr[12]_i_5*) (au total 4 LUT4, 1 LUT5 et 2 LUT6). À noter qu'un unique signal de comparaison, équivalent à *end_counter* n'est pas créé : lorsque la comparaison est vraie la sortie de *current_state_i_2* vaut '0' et celle de *current_state_i_3* vaut '1', et c'est ce duo de signaux qui sera évalué dans les futurs tests.

À noter : étrangement le 10^{ème} bit du compteur apparaît deux fois dans la comparaison globale, ce qui semble redondant.

- Une table de vérité à 3 entrées LUT3 (*current_state_i_1*) utilise le duo de signaux décrit précédemment et l'ancienne valeur de *current_state* pour déterminer sa nouvelle valeur : lorsque le duo de signaux équivaut au compteur qui a atteint sa cible, la valeur de *current_state* est inversée.
- Le réinitialisation et l'incréméntation du dernier bit de *Reg_ctr* est géré par une table de vérité à 3 entrées LUT3 (*Reg_ctr[0]_i_3*), qui prend en entrée le duo de signaux et la valeur de *Reg_ctr[0]* : lorsque le duo de signaux équivaut à une fin de compteur, la sortie est réinitialisée à '1', sinon le dernier bit est inversé (équivalent d'une incréméntation).
- L'éventuelle propagation de l'incréméntation du dernier bit sur les bits supérieurs est gérée par une autre LUT3 (*Reg_ctr[0]_i_2*), avec les mêmes entrées que précédemment. La sortie de la LUT vaut la même valeur que le dernier bit de *Reg_ctr*, sauf si le duo de signaux équivaut à une fin de compteur, et dans ce cas la sortie vaut '1'. Cette valeur servira d'entrée DI[0] à une primitive CARRY4 (voir schéma), et comme on l'avait noté dans le TP2, la LUT entière peut apparaître comme superflue car en cas de fin de compteur l'entrée S[0] du CARRY4 vaut '1' (*Reg_ctr[0]* est réinitialisé à '1') et l'entrée DI[0] n'est pas prise en compte, à la place de la LUT, un simple lien vers la sortie de *Reg_ctr_reg[0]* suffirait.



- Pour les bits supérieurs, on a vu dans le TP2 qu'on peut pour ce compteur remplacer l'opération « incrémentation ou réinitialisation » par « réinitialisation puis incrémentation » :

- La réinitialisation est normalement effectuée par une LUT3, qui prend en entrées le duo de signaux indiquant la fin de compteur et la valeur de *Reg_ctr_reg* du bit concerné. En cas de fin de compteur, la sortie de la LUT est remise à '0', sinon on conserve la valeur en sortie du registre.

Nous pouvons noter que nous observons dans cette synthèse une nouvelle optimisation : seuls les bits qui valent '1' lorsque le compteur atteint la *cible* sont concernés par une LUT effectuant un reset. Les autres bits n'ont pas besoin de réinitialisation, la valeur en sortie du registre est directement utilisée.

- Cependant, la réinitialisation des bits 12, 20 et 25 suit une logique différente : au lieu d'utiliser une LUT3 et le duo de signaux *current_state_i_2* et *current_state_i_3*, chacun de ces bits recalcule avec sa propre LUT6 (*Reg_ctr[12]_i_3*, *Reg_ctr[20]_i_4* et *Reg_ctr[24]_i_3*) une partie de la comparaison avec la *cible*, en utilisant comme entrée les registres des bits 12, 20 et 25, et les sorties des LUT *current_state_i_2*, *Reg_ctr[12]_i_5* et *Reg_ctr[12]_i_4* (cette dernière est une LUT5 spécifique qui recalcule une partie des bits de la comparaison, et qui est donc redondante).
- La propagation de l'incrémement est faite dans des CARRY4, chacune fonctionnant sur 4 bits, pour un total de **7 CARRY4**.

Au total, cette sous-partie utilise 27 FDCE, 13 LUT3, 4 LUT4, **2 LUT5** et **5 LUT6**. Avec les primitives décrites dans le reste du schéma, on obtient bien un total de **17 LUT3** et **5 LUT4**.

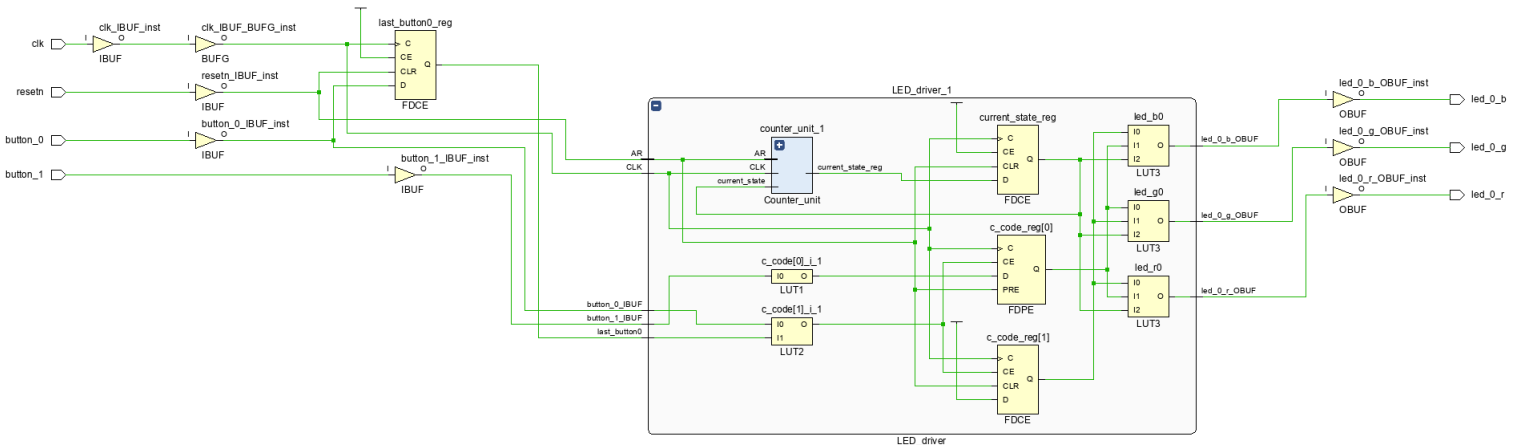
Puisque cette synthèse aboutit à plusieurs redondances dans son résultat sur la partie *Counter_unit*, nous allons essayer d'avoir une solution plus efficace sur ses ressources. Nous relançons une synthèse, cette fois avec la stratégie « *Flow_AreaOptimized_high* ».

Le rapport de synthèse (disponible dans son entièreté dans le fichier *LED_module_synth2.vds*) nous donne maintenant les ressources utilisées suivantes :

Report Cell Usage:			
	Cell	Count	
1	BUFG	1	
2	CARRY4	7	
3	LUT1	1	
4	LUT2	17	
5	LUT3	3	
6	LUT4	3	
7	LUT5	3	
8	LUT6	1	
9	FDCE	30	
10	FDPE	1	
11	IBUF	4	
12	OBUF	3	

Sans entrer pour l'instant dans le détail, on observe une économie d'une LUT par rapport à la première synthèse.

Le schéma correspondant à cette nouvelle synthèse, non-entièrement développé encore une fois, est le suivant :



Le schéma développé est disponible en entier dans le fichier *Schematique_TP4-1_synth2.pdf*.

Les différences avec la solution de synthèse précédente sont les suivantes :

- L'actualisation du code couleur interne *c_code* est toujours géré par deux LUT, mais plus bit par bit (avec une LUT3 et une LUT4) :
 - une LUT2 (*c_code[1]_i_1*) calcule l'équivalent du signal *update* en détectant le front montant de *button_0*, et ce signal pilote l'entrée enable CE des registres *c_code_reg[0]* et *c_code_reg[1]* autorisant une mise à jour des registre uniquement en cas de signal *update*.
 - Une LUT1 (*c_code[0]_i_1*) calcule la mise à jour du dernier bit de *c_code* en fonction de l'état de *button_1*.
 - Le registre du premier bit de *c_code* n'a pas d'entrée variable, sa valeur est toujours haute : la valeur du bit est basse à l'initialisation et après reset, et haute après un premier signal *update*.
- Le signal *end_counter* est calculé par 7 LUT, **3 LUT4**, **3 LUT5** et **1 LUT6** (de *current_state_i_2* à *current_state_i_8*), la valeur étant directement accessible en sortie de *current_state_i_2*.
- Le signal *current_state* est calculé par une LUT2 (*current_state_i_1*) à partir de *end_counter* et de l'ancienne valeur de *current_state*.
- Le dernier bit de *Reg_ctr* et l'équivalent au signal *end_counter* sont utilisés en entrée de 2 LUT2 : *Reg_ctr[0]_i_3* qui gère l'incréméntation (inversion) et la réinitialisation du bit, et *Reg_ctr[0]_i_2* qui gère la propagation de l'incréméntation aux bits supérieurs.
- Tous les bits supérieurs de *Reg_ctr* qui nécessitent une réinitialisation en fin de compte sont maintenant gérés par des LUT2. Les redondances de comparaison pour les bits 12, 20 et 25 ont disparu.

Cependant, le nombre de LUT total utilisé comparé dans ces deux descriptions pour les deux synthèses est un nombre de primitives, et non de « slice » de LUT physiques, qui est la ressource limitée sur un FPGA. En effet, deux LUT ayant un total de 5 entrées différentes ou moins peuvent se combiner sur une unique LUT6_2, qui est la seule type de LUT gravée sur un FPGA Xilinx Series7.

En pratique, les rapports d'utilisation de ces deux synthèses (fichiers *LED_module_utilization_synth1.rpt* et *LED_module_utilization_synth2.rpt*) indiquent un nombre de « slice » LUT de 25 pour la première, et de 26 pour la seconde, malgré le nombre réduit de primitives pour la seconde.

Au final, de telles considérations d'optimisation n'ont pas beaucoup d'importance dans un contexte où l'on est très loin des limites de ressources en LUT. Pour notre Cora Z7 qui possède 17 600 LUT, on utilise avec ce design moins de 0,2 % de ses ressources. Pour la suite, on continuera en repartant du résultat de la première synthèse.

Avant de passer à l'implémentation dans la question suivante, on va également configurer les sondes ILA pour le debug de notre design.

On suivra les signaux suivants :

- Les 2 entrées des boutons 0 et 1 (*button_0_IBUF* et *button_1_IBUF*)
- Les 3 sorties LED (*led_0_r_OBUF*, *led_0_g_OBUF* et *led_0_b_OBUF*)
- Les 2 bits du signal *c_code* (*c_code[0]* et *c_code[1]*)
- L'état de la FSM (*current_state*)
- Les 2 signaux équivalent à *end_counter* (*current_state_i_2_n_0* et *current_state_i_2_n_1*)

Les contraintes liées à ce module de debug sont présentes dans le fichier *Cora-Z7-10-TP4-1-Debug.xdc*. Les contraintes liées au placement des autres entrées et sorties sur la carte sont dans le fichier de contraintes *Cora-Z7-10-Master.xdc*. À noter : en l'absence d'entrée dédiée pour le signal *resetsn*, il a été placé sur le premier pin du connecteur Pmod JA.

14) On effectue l'implémentation du premier résultat de synthèse.

Le processus d'implémentation ne soulève aucune erreur ou critical warning.

Analysons le rapport de timing (le rapport complet est disponible dans le fichier *LED_module_timing_summary_routed.rpt*).

Le récapitulatif de timing obtenu est le suivant :

Design Timing Summary											
WNS(ns)	TNS(ns)	TNS Failing Endpoints	TNS Total Endpoints	WHS(ns)	THS(ns)	THS Failing Endpoints	THS Total Endpoints	WPWS(ns)	TPWS(ns)	TPWS Failing Endpoints	TPWS Total Endpoints
1.904	0.000	0	4239	0.029	0.000	0	4223	2.750	0.000	0	2415

On observe notamment que :

- la valeur de *Worst Negative Slack (WNS)* vaut 1,904 ns, ce qui est positif et indique qu'il n'y a pas de violation de set up.
- La valeur de *Worst Hold Slack (WHS)* vaut 0,029 ns, ce qui est positif et indique qu'il n'y a pas de violation de hold.

Le récapitulatif d'horloge est le suivant :

Clock Summary			
Clock	Waveform(ns)	Period(ns)	Frequency(MHz)
dbg_hub/inst/BSCANID.u_xsdbrm_id/SWITCH_N_EXT_BSCAN.bscan_inst/SERIES7_BSCAN.bscan_inst/TCK	{0.000 16.500}	33.000	30.303
sys_clk_pin	{0.000 4.000}	8.000	125.000

L'horloge système *sys_clk_pin* est bien configurée selon nos spécifications initiales : période de 8 ns (soit une fréquence de 125 MHz), et signal carré (même demi-période de 4 ns pour les crêtes haute et basse du signal).

Le chemin critique présenté par le rapport est le suivant :

```
From Clock: sys_clk_pin
To Clock: sys_clk_pin

Setup :      0 Failing Endpoints, Worst Slack      1.904ns, Total Violation      0.000ns
Hold :       0 Failing Endpoints, Worst Slack      0.029ns, Total Violation      0.000ns
PW :        0 Failing Endpoints, Worst Slack      2.750ns, Total Violation      0.000ns

Max Delay Paths
-----
Slack (MET) :      1.904ns (required time - arrival time)
Source:      <hidden>
              (rising edge-triggered cell FDRE clocked by sys_clk_pin {rise@0.000ns fall@4.000ns period=8.000ns})
Destination: <hidden>
              (rising edge-triggered cell FDRE clocked by sys_clk_pin {rise@0.000ns fall@4.000ns period=8.000ns})
Path Group:   sys_clk_pin
Path Type:    Setup (Max at Slow Process Corner)
Requirement:  8.000ns (sys_clk_pin rise@8.000ns - sys_clk_pin rise@0.000ns)
Data Path Delay: 5.888ns (logic 0.774ns (13.144%) route 5.114ns (86.856%))
Logic Levels: 1 (LUT6=1)
Clock Path Skew: -0.203ns (DCD - SCD + CPR)
  Destination Clock Delay (DCD):  4.839ns = ( 12.839 - 8.000 )
  Source Clock Delay (SCD):  5.318ns
  Clock Pessimism Removal (CPR):  0.276ns
Clock Uncertainty: 0.035ns ((TSJ^2 + TIJ^2)^1/2 + DJ) / 2 + PE
  Total System Jitter (TSJ):  0.071ns
  Total Input Jitter (TIJ):  0.000ns
  Discrete Jitter (DJ):  0.000ns
  Phase Error (PE):  0.000ns
```

On remarque que le slack de ce chemin critique est à environ 24 % d'une période. C'est supérieur à la marge de 20 % qu'on veut avoir pour s'assurer d'un bon fonctionnement de notre système.

15) Sur Vivado, on génère le bitstream correspondant à notre implémentation, et on programme notre carte avec lui.

La vidéo *Démo_LED_module.mp4* présente 15 secondes de fonctionnement de la carte programmée.

La vidéo commence alors que la carte est dans son état initial (clignote en rouge).

Pendant les 2 premières secondes, on observe le clignotement en rouge de la LED.

À la 2^{ème} seconde, on appuie rapidement sur le bouton 0 sans appuyer sur le bouton 1 : la LED passe du rouge au bleu. Elle continue à clignoter en bleu à partir de là.

À la 6^{ème} seconde, on commence à appuyer sur le bouton 1, sans toucher à l'autre. Pour l'instant, le comportement de la LED est inchangé, elle continue de clignoter en bleu.

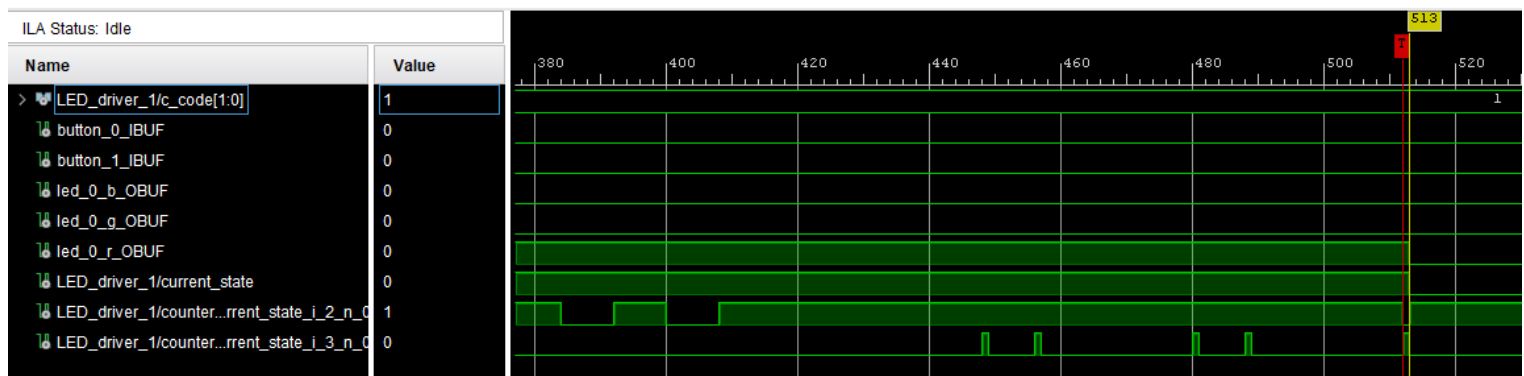
À la 8^{ème} seconde, on appuie rapidement sur le bouton 0, alors que le bouton 1 est lui aussi appuyé : la LED passe du bleu au vert. Elle continue de clignoter en vert après cela.

À la 9^{ème} seconde, on relâche le bouton 1, sans effet sur le comportement de la LED.

À la 12^{ème} seconde, on appuie rapidement sur le bouton 0 alors que le bouton 1 est toujours relâché : la LED repasse du vert au bleu, et continue de clignoter en bleu jusqu'à la fin de la vidéo.

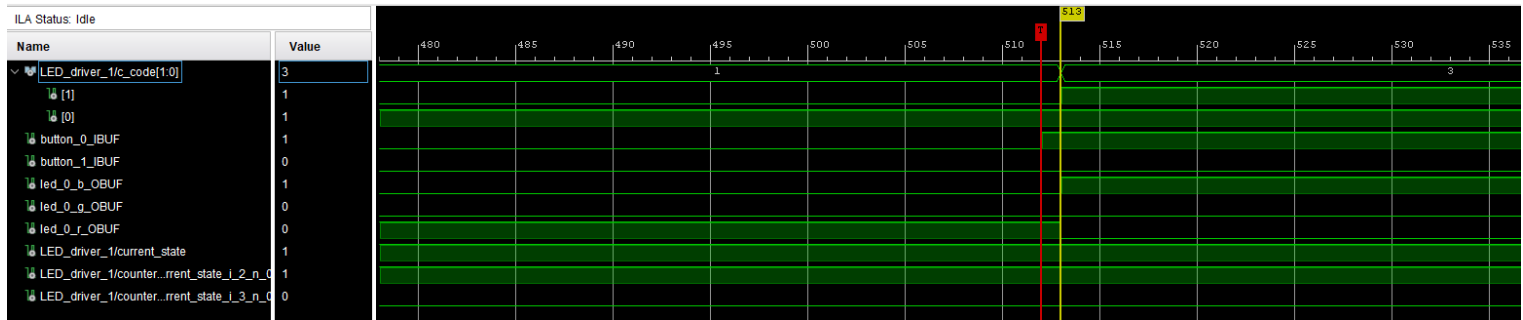
En plus de cette évaluation visuel, on effectue plusieurs tests avec la ILA.

Le premier test vérifie le fonctionnement du clignotement de la LED, après un reset lorsque la LED clignote en rouge. Il est trigger par l'équivalent du signal *end_counter*, c'est à dire quand *current_state_i_2_n_0* vaut '0' et *current_state_i_3_n_0* vaut '1'.



Si l'on peut voir individuellement le signal *current_state_i_2_n_0* prendre la valeur '0' et le signal *current_state_i_3_n_0* prendre la valeur '1' avant le trigger, c'est bien quand le duo vaut respectivement '0' et '1' que l'équivalent du signal *end_counter* est activé, ce qui provoque au coup d'horloge suivant l'inversion du signal *current_state* (qui passe dans ce cas de '1' à '0') et le changement d'état de la LED (ici une extinction sur le signal *led_0_r_OBUF*).

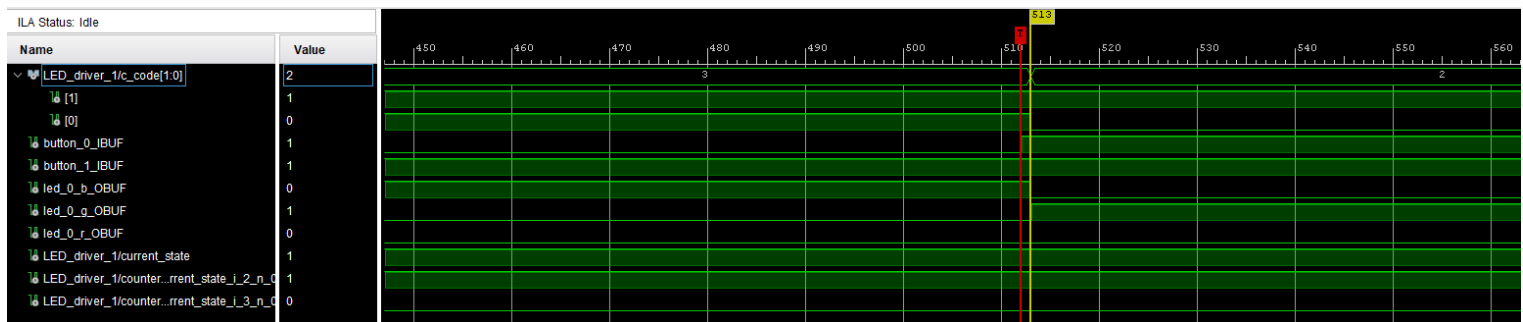
Le second test, lancé après un reset de la carte alors que la LED est en train de clignoter en rouge, est trigger par un front montant de l'entrée *button_0_IBUF* (signal *update*) :



Comme on l'avait simulé, le changement du code couleur *c_code* a lieu un coup d'horloge après le front montant du bouton 0.

Pendant ce test, aucune condition n'a été mis sur l'état du bouton 1, mais on peut observer qu'il n'était pas appuyé (*button_1_IBUF* est à '0' pendant tout le test), le changement de *c_code* à donc lieu vers la valeur '11' qui correspond au bleu, et on observe le changement de couleur de la LED (de rouge vers bleu, *led_0_r_OBUF* passe à '0' alors que *led_0_b_OBUF* passe à '1') au moment du changement de *c_code*.

Le troisième test est lancé alors que la carte est en train de clignoter en bleu. Il est trigger par un front montant de l'entrée *button_0_IBUF* (signal *update*) alors que *button_1_IBUF* vaut '1' (bouton 1 appuyé) :

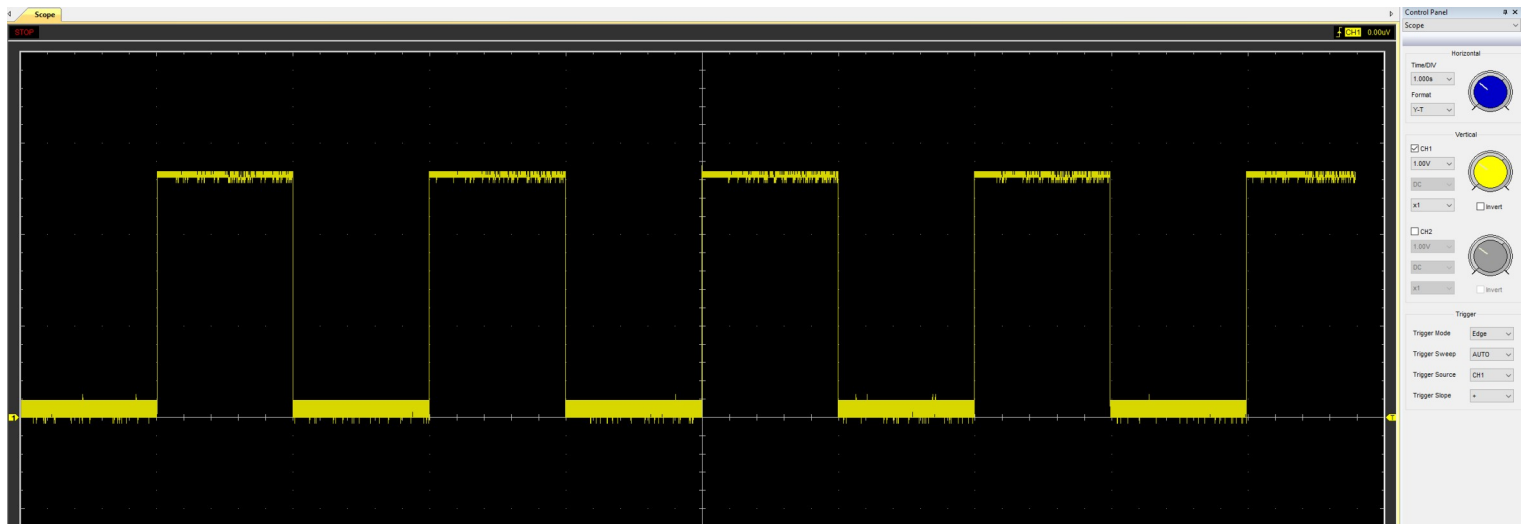


Un coup d'horloge après le front montant du bouton 0, le code couleur interne *c_code* change, cette fois pour prendre la valeur '10', et on observe qu'à ce moment la LED passe du bleu au vert (*led_0_b_OBUF* passe de '1' à '0' alors que *led_0_g_OBUF* passe de '0' à '1').

Enfin, on vérifie également la durée de nos cycles avec une mesure à l'oscilloscope.

Notre mesure est effectué au pin d'entrée de la LED bleue. Selon la documentation de notre carte Cora Z7, le signal de sortie de FPGA qui pilote les LED est inversé : lorsque le signal de sortie est haut, il est alors bas en sortie de la LED qui se retrouve sous tension, et donc allumée.

Nous mesurons le signal suivant (division verticale de 1 V, division horizontale de 1 s) :



On observe bien que lorsque la LED bleue est éteinte, le signal à l'entrée de la LED est haut, ce qui correspond à un signal de sortie bas, et inversement quand la LED est allumée. Notre carte semble avoir perdu un peu de puissance, puisque le signal haut monte vers 2,6 V, et non les 3,3 V prévu pour du LVCMOS33.

Sur cet enregistrement de notre signal, on observe que la période allumée de la LED, ainsi que la période éteinte entre deux clignotements valent bien 1 s à chaque fois. Cela confirme notre réglage du couple horloge – paramètre de *cible* de notre *Counter_unit* pour obtenir des cycles de 1 seconde.

Au final, que ce soit en visuel, via l'ILA ou à l'oscilloscope, la carte semble parfaitement fonctionner.